

(Optional) HTML Routes and Controllers

The same routes, now answering with HTML pages.

Learning Goals

- Distinguish `routes/web.php` from `routes/api.php`, and a web controller from an API controller.
- Return an HTML page and handle a form with a GET route and a POST route.
- Explain CSRF tokens and output escaping, and why a browser form sends only GET and POST.

In `3-1`, the controller answered with **JSON** through `routes/api.php`. A person in a browser needs **HTML** instead.

- Laravel has a second route file for this: `routes/web.php`.
- The idea is the same: HTTP method + path → controller method → response.
- Only the response changes: a string of HTML instead of JSON.
- You can skip this lesson. The rest of the module builds APIs.

Two Route Files, Two Controllers

	API (3-1)	Web (this lesson)
Route file	<code>routes/api.php</code>	<code>routes/web.php</code>
URL	<code>/api/student</code>	<code>/student</code>
Controller	<code>Api\StudentController</code>	<code>StudentController</code>
Returns	<code>response()->json(...)</code>	an HTML string
Middleware	API: no session	Web: sessions and CSRF protection
Client	<code>curl</code> , apps	Browser

- Both controllers have the same method names (`show` , `store` , `hello ...`). They live in different folders, so their namespaces differ

The Same Method, Two Answers

```
// Api\StudentController
public function show()
{
    return response()->json($this->getStudent());
}
```

```
// StudentController (web)
public function show()
{
    $student = $this->getStudent();
    return '<h1>Student Information</h1>
        <p>Name: ' . $student['name'] . '</p>';
}
```

- The data is the same. The controller decides how to present it.

The Web Routes

routes/web.php :

```
Route::get('/student', [StudentController::class, 'show']);  
Route::get('/student/create', [StudentController::class, 'create']);  
Route::get('/student/{id}', [StudentController::class, 'showById']);  
Route::post('/student', [StudentController::class, 'store']);  
Route::get('/hello', [StudentController::class, 'hello']);  
Route::get('/time', [StudentController::class, 'time']);
```

- `{id}` is a **route parameter**: `/student/5` calls `showById('5')`.
- `/student/create` must come **before** `/student/{id}`.
Laravel uses the first match, and `create` would otherwise be read as an id.

A Form Page and Its POST

```
public function create()
{
    return '<form method="POST" action="/student">' . csrf_field() . '
        <p>Name: <input type="text" name="name" required></p>
        <p><button type="submit">Create Student</button></p>
        </form>';
}

public function store(Request $request)
{
    return '<p>Name: ' . e($request->name) . '</p>';
}
```

- The first route (GET) **shows** the form. The second route (POST) **receives** it.
- `$request->name` reads the field named `name` .

Two Things Every Web Form Needs

- `csrf_field()` adds a hidden **CSRF token**. A web `POST`, `PUT`, or `DELETE` without it gets **419 Page Expired**. API routes do not need it.
- `e($request->name)` escapes user text. A name like `<script>` is shown as text and does not run in the browser.

```
# A POST without the token is rejected:  
curl -i -X POST http://localhost:8080/greet -d 'name=Jane'      # 419
```

Only GET and POST in a Browser

The web routes also define `PUT /student` and `DELETE /student`.

- An HTML form can send only `GET` and `POST`. A browser cannot send `PUT` or `DELETE` from a plain form.
- In this lesson, those two routes show the syntax. Test them with `curl` or use the API routes.
- Lesson 6 shows the Laravel way: a hidden `_method` field (`@method('PUT')`) in `6-3`.

Run the Code

```
cd "6. View and Controllers/6-1"  
bash ./run1-6-1.sh  
cd student-api  
php artisan serve --port=8080  
# WSL2: php artisan serve --host=0.0.0.0 --port=8080
```

- The script copies the API files from `3. Routes and Controllers/student-api1-3` and the HTML files from `6-1/student-api1-6-1` .
- So the Lesson 3 folder must exist. The script stops with a message if it does not.
- This lesson has no database, so the script does not touch MySQL.

Try It in the Browser

URL	What you see
<code>http://localhost:8080/</code>	A page with links to all the examples
<code>/student</code>	The fixed student as HTML
<code>/student/5</code>	The id from the URL
<code>/student/create</code>	A form. Submit it to reach <code>POST /student</code>
<code>/hello</code> , <code>/time</code>	Simple pages
<code>/greet</code>	A form that greets you by name
<code>/api/student</code>	The same student as JSON

Check the Routes

```
php artisan route:list --except-vendor
```

```
GET|HEAD  /  
GET|HEAD  student ..... StudentController@show  
POST      student ..... StudentController@store  
GET|HEAD  api/student ..... Api\StudentController@show  
POST      api/student ..... Api\StudentController@store
```

- The `api/` prefix comes from `routes/api.php`. The web routes have no prefix.
- `StudentController` and `Api\StudentController` are two different classes.

Why This Part Is Optional

The HTML pages are extra. You can skip them.

- The rest of this module builds APIs, so Lessons 1 to 5 are enough to continue.
- Use this part to see the difference between web routes and API routes, and to meet CSRF early.
- Next: **6-2** adds a database to these pages. **6-3** replaces the HTML strings with Blade templates.